# SECURITY

Mei Nagappan

UNIVERSITY OF WATERLOO | FACULTY OF MATHEMATICS

# Guideline 4: Prompt for Hardened Infrastructure Configs and Verify Before Deployment

AI-scaffolded apps generate Docker Compose, database configs, and cloud templates with insecure defaults : open ports, missing auth, wildcard firewall rules.

Write infrastructure-specific prompts that name security constraints explicitly.

Review every AI-generated infrastructure config for network exposure, authentication, and permissions before deploying.

# Guideline 4: Why This Matters

- OpenSSF flags AI-generated IaC as high-risk: wildcard IAM policies and public-by-default resources are recurring patterns
- AI-generated Terraform routinely deploys resources publicly accessible because the model copies permissive training examples
- These flaws pass all application-level SAST scans - Bandit checks Python, not whether your database port is exposed to the internet
- Naming security constraints explicitly in the prompt significantly reduces vulnerabilities in generated output

# Guideline 4: Prompting for Secure Infrastructure

- State the deployment environment explicitly: 'this will run on a public cloud VM'
- Name infrastructure constraints: 'do not bind any database port to 0.0.0.0', 'run all containers as a non-root user'
- Ask for least privilege: 'follow principle of least privilege for all IAM roles, security groups, and firewall rules'
- Ask for a .env pattern: 'externalize all credentials to environment variables'
- Ask the model to justify each exposed port and permission: 'explain why it is necessary'

# Guideline 4: Reviewing Generated Infrastructure

- Check every database config for the bind address: 0.0.0.0 means publicly accessible; it must be 127.0.0.1 or a private interface
- Verify authentication is enabled and default credentials (root, admin, password) have been replaced
- Review every Docker Compose manifest: '5432:5432' on a cloud VM with no firewall is a public database
- Audit every IAM role and security group for wildcard permissions (*) or open CIDR ranges (0.0.0.0/0)
- Run Checkov or docker-bench-security on every AI-generated config before deployment

# Guideline 5: Verify Every AI-Suggested Dependency Before Installing

Before running any pip install or npm install suggested by an AI tool, manually confirm the package exists on the official registry.

Check for recent maintenance activity and known CVEs.

Run pip audit or npm audit after every install.

# Guideline 5: Why This Matters

- 'Slopsquatting' (2025): attackers register PyPI/npm packages matching names that AI models commonly hallucinate
- Models pre-trained on older codebases suggest libraries that have since accumulated critical CVEs
- Both attacks share the same mitigation: verify on the official registry and run an analysis tool before use

# Guideline 5: Key Practices

- Ask the model to use only well-known packages with over 1 million monthly downloads and active maintenance
- Ask the model to show the install command with a pinned version alongside the code so you can audit before running
- If unsure whether a package exists, ask: 'Does this package exist on PyPI/npm? What is its homepage?'
- Search every AI-suggested package name on the official registry before installing: a 404 is a red flag
- Run pip audit or npm audit immediately after install and before running any code
- Pin verified, CVE-free versions explicitly in requirements.txt or package.json
- Enable Dependabot or Snyk on every repository to catch vulnerabilities that emerge after merge

# Guideline 6: Anchor LLM Vulnerability Review with SAST Tool Output

Instead of asking the AI to 'review this code for security issues', run a static analysis tool first and paste its output into the prompt.

SAST output (CWE ID, line number, severity) grounds the model on a specific, verifiable finding.

This prevents security degradation from unanchored LLM feedback loops.

# Guideline 6: Why This Matters

- CodeQL-grounded prompts reduced residual vulnerability rate from 40.2% to 7.4% for GPT-4
- Unanchored LLM iteration increased critical vulnerabilities by 37.6% after five rounds
- Asking the model to 'make it more secure' without a failing test gives no deterministic signal
- The model must be anchored to a re-runnable oracle , not left to free-form self-review

# Guideline 6: Prompting

- Always run Bandit, Semgrep, or CodeQL before asking the model to fix a vulnerability
- Paste the tool's full output (CWE ID, line number, severity, description) verbatim into the prompt
- Ask the model to explain why the flagged line is vulnerable before asking it to produce a fix
- After the fix is generated, re-run the same tool and paste the new output back; do not iterate more than twice without a human review checkpoint

# Guideline 6: Reviewing the Result

- Re-run the SAST tool on the fixed code before accepting the output - do not rely on the model's claim that the issue is resolved
- Check that the fix does not introduce a new vulnerability in an adjacent line (a common failure mode in unanchored iteration)
- Confirm the fix is logically equivalent to the original - refactors that change behavior are a separate review item

# References


[1] Ullah et al. (2024). LLMs Cannot Reliably Identify and Reason About Security Vulnerabilities (Yet?). arXiv:2312.12575.

[2] Tony et al. (2024). Prompting Techniques for Secure Code Generation: A Systematic Investigation. arXiv:2407.07064.

[3] OWASP (2021). Threat Modeling Cheat Sheet. cheatsheetseries.owasp.org

[4] NIST (2022). Secure Software Development Framework (SSDF), SP 800-218. doi.org/10.6028/NIST.SP.800-218

[5] OpenSSF Best Practices Working Group (2025). Security-Focused Guide for AI Code Assistant Instructions.

# References (continued)

[6] AppSec Engineer (2025). AI-Generated Code Needs Its Own Secure Coding Guidelines. appsecengineer.com

[7] arXiv 2506.23034 (2025). Guiding AI to Fix Its Own Flaws: An Empirical Study on LLM-Driven Secure Code Generation.

[8] Ioannidis et al. (2025, IEEE-ISTAS). Security Degradation in Iterative AI Code Generation. arXiv:2506.11022.

[9] Soares et al. (2024). Property-Based Testing in Practice. ICSE 2024.